

drop

Descripción de la Aplicación	2
Características principales:	2
Estructura del Proyecto	3
Panel Eventos	6
Crear	7
Mostrar	8
Click al JTable	9
Editar	9
Eliminar	10
Ver Categoría	11
Crear Categoría	12
Eliminar Categoría	12
Panel Reservas	13
Hacer Reserva	16
Ver Reservas	18
Hilos	19

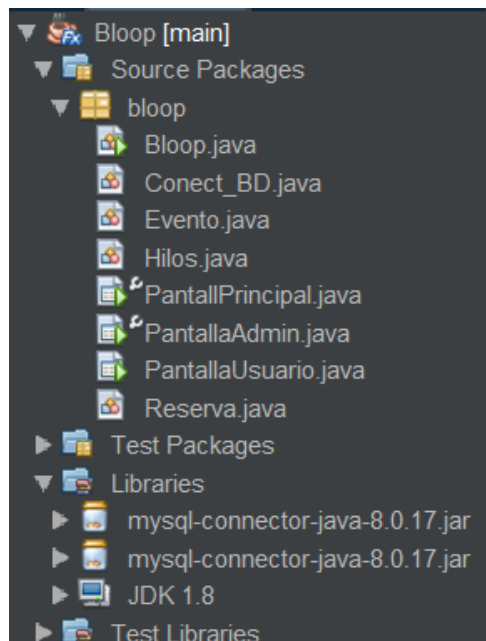
Descripción de la Aplicación

Esta aplicación es un sistema de **reserva de eventos** desarrollada en JavaFX, diseñada para facilitar la gestión y organización de eventos a través de una interfaz intuitiva y funcional. El sistema permite a los usuarios, iniciar sesión y realizar reservas de eventos disponibles, mientras que los administradores pueden gestionar dichos eventos y monitorear las reservas realizadas.

Características principales:

- **Inicio de sesión seguro:** el sistema cuenta con un mecanismo de autenticación que permite el acceso diferenciado para usuarios y administradores.
- **Pantalla de usuario:** una vez autenticado, el usuario tiene acceso a una interfaz donde puede:
 - Ver los eventos disponibles
 - Realizar reservas
 - Consultar sus reservas anteriores
- **Pantalla de administrador:** los administradores acceden a una vista exclusiva que les permite:
 - Crear, modificar y eliminar eventos
 - Visualizar todas las reservas realizadas





Estructura del Proyecto

El proyecto incluye tres pantallas principales (JFrame):

- **Inicio de sesión**
- **Usuario**
- **Administrador**

La conexión a la base de datos está manejada por la clase **Conect_BD**.

Las clases principales son:

- **Reserva** (gestiona las reservas)
- **Eventos** (gestiona los eventos)

Además, hay una clase **Hilos** para manejar tareas en segundo plano mediante multihilos.

Para poder conectar la base de datos al código hemos utilizado la Librería mysql-connector-java-8.0.17.jar y JDK 1.8 como herramientas necesarias para desarrollar, compilar y ejecutar aplicaciones en Java para realizar JavaFX

Pantalla Principal

En esta pantalla principal su único funcionamiento es el **Inicio de Sesión**, tanto como Usuario y Administrador.

¿Cómo lo hacemos? Primero hay que hacer una conexión a la base de datos mediante un método (El código insertado abajo).

Consiste en que mediante el método, recibimos un JTextField (Nombre del Usuario) y un JPasswordField (Contraseña), mediante esa información realizamos nuestra consulta en la base de datos para primero recibir el ID del usuario y su Nivel de acceso.

Estos parámetros los recuperamos en unas variables para luego jugar con ellas en otros sectores de la aplicación.

```
1 public void Inicio_Sesion(JTextField JT, JPasswordField JP) {
2
3     //LO QUE HACEMOS AQUI ES QUE LE PEDIMOS UN FIELD Y UN PASSWORD ASI HACEMOS LA CONSULTA
4     try {
5         if (!JT.getText().equals("") || !JP.getPassword().equals("")) {
6             Class.forName("com.mysql.cj.jdbc.Driver");
7             cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/bloop", "root", "");
8
9             String SQL = "SELECT Id_usuario, nivel_acceso FROM usuario WHERE Nombre = ? AND Contraseña = ?";
10            PreparedStatement ps = cn.prepareStatement(SQL);
11            ps.setString(1, JT.getText());
12            ps.setString(2, new String(JP.getPassword())); // Convierte el password char[] a String
13
14            rs = ps.executeQuery();
15
16            //SI LA CONSULTA A DADO RESULTADO NO HACE EL IF
17            if (rs.next()) {
18                //QUE NOS METE EL BOOLEAN DE EL NIVEL EN UNA VARIABLE Y LA SACAMOS FUERRA
19                resultado = String.valueOf(rs.getBoolean("nivel_acceso"));
20                ID = rs.getInt("Id_usuario");
21                System.out.println(resultado);
22            } else {
23                System.out.println("Usuario y contraseña Incorrecto");
24            }
25
26        } else {
27            System.out.println("DEBE COMPLETAR LOS CAMPOS");
28        }
29    }
30 }
```



Después de realizar la conexión a la base de datos nos vamos a nuestra pantalla principal, la cual tenemos el JText y el Password para que introduzca su usuario y contraseña.

En el código del botón lo primero de todo es creando el objeto de la conexión e inicializamos el método `Iniciar_Sesion` (Código de arriba). Después de haber inicializado realizamos un IF que nos comprueba si los campos están en blanco si es así nos saca un `JOptionPane` de que debemos rellenar los campos. Si el caso es contrario y hay campos recopilamos el campo de **Nivel Acceso**(booleano) y en nuestro caso **0(false)** es Usuario u **1(true)** es administrador.

En el caso de Administrador nos abrirá la Pantalla de Administrador, mientras que en el caso de Usuario nos abre la Pantalla de Usuario, además de pasar el ID del usuario que está accediendo.

```
1 private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_jButton1ActionPerformed
2     objConect_BD.Inicio_Sesion(jTextField1, jPasswordField1);
3
4     //GENERAMOS EL OBJETO AÑADIENDO EL FIEL DEL NOMBRE Y EL PASSWOR DE LA CONRSASEÑA
5     //HACEMOS UN IF QUE SI NO HAY NADA ESCRITO NOS SAQUE UN MENSAJE DE QUE RELLENE LOS CAMPOS
6
7     if (!jTextField1.getText().equals("") || !jPasswordField1.getText().equals("")) {
8         //SI NO ES LE CASO DE QUE LOS CAMPOS NO ESTAN VACION NOS COMPRUEBA LA VARIABLE DEL BOOLEAN QUE SACAMOS ANTES SI ES TRUE
9         //SIGNIFICA QUE ES ADMINSTRADOR Y LE ABRE LA PESTAÑA DEL ADMINITRADOR
10        if (objConect_BD.resultado.equals("true")) {
11            setVisible(false);
12            Obj_pantallaAdmin.setVisible(true);
13        }
14        //SI ES FALSE SIGNIFICA QUE ES USUARIO Y NOS HABRE LA VENTANA DE USUARIO
15        if (objConect_BD.resultado.equals("false")) {
16            setVisible(false);
17            obj_pantauser.setVisible(true);
18            obj_pantauser.ID(objConect_BD.ID);
19        }
20    }
21    //SI NO DUELVE NADA ES EL CASO DE QUE ES INCORRECTO NOS SACA UN MENSAJE DE QUE ES INCORRECTO
22    if (objConect_BD.resultado.equals("")) {
23        jPasswordField1.setText("");
24        JOptionPane.showMessageDialog(null, "El Usuario o Contraseña es incorrecta", "Bloop", JOptionPane.INFORMATION_MESSAGE);
25    }
26    }else{
27        JOptionPane.showMessageDialog(null, "Debes rellenar los campos, para poder acceder", "Bloop", JOptionPane.INFORMATION_MESSAGE);
28    }
29 }
```

Pantalla Administrador



En nuestra pantalla administrador las funciones principales son las siguientes:

- Panel Evento
- Panel Reservas

Panel Eventos

ID	Categoria	Nombre	Fecha	Ubicacion	Plazas Totales	Plazas Disponibles
----	-----------	--------	-------	-----------	----------------	--------------------

Esta es nuestra pantalla como puedes ver tenemos un Jtable donde muestra la información y en la parte de la izquierda todo las acciones que puede hacer tanto el mostrar, editar, crear, editar y el crear nueva categoría y eliminarla.

Crear

Para el botón de crear primero es crear un objeto de Evento y que previamente hemos creado y lo rellenamos con los destinos JText, JCombo, JSpinner que es lo que coloca el administrador para crear un nuevo evento.

En el SetPlazas_Disponibles al ser creado las plazas en ese momento serán las que administrador ha colocado para las Plazas Totales.

Y por último ejecutamos el método de crear evento del Objeto de la base de datos

```
1 private void jButton9ActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_jButton9ActionPerformed
2     Evento obj_eventos = new Evento();
3
4
5     obj_eventos.setCategoria(jComboBox1.getSelectedItem().toString());
6     obj_eventos.setTitulo(jTextField2.getText());
7     obj_eventos.setFecha(Date.valueOf(jTextField3.getText()));
8     obj_eventos.setUbicacion(jTextField4.getText());
9     obj_eventos.setPlazas_Totales((Integer) jSpinner1.getValue());
10    obj_eventos.setPlazas_Disponibles((Integer) jSpinner1.getValue());
11
12    objCont.Crear_Evento(obj_eventos);
13 }
```

En el objeto Crear_Evento pedimos un objeto Evento que es el que pasamos en las pantalla.

Iniciamos la conexión a la base de datos, Creamos la consulta de INSERT ya que vamos a crear con una subconsulta ya que solo nos dan el nombre de la categoría y para crear necesitamos el Id.

Introducimos la información en sus respectivas posiciones dependiendo de donde este el ? antes introduciendo la SQL en la base de datos y ejecutamos con executeUpdate (aquí yo lo meto en un int para ver las filas que están afectadas esto viene bien para ver si hay algún error y alomejor se crean 2)

```
1 public void Crear_Evento(Evento EV) {
2     try {
3         Class.forName("com.mysql.cj.jdbc.Driver");
4         cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/bloop", "root", "");
5
6         String SQL = "INSERT INTO evento (Id_Categoria_Evento, Titulo, Fecha, Ubicacion ,Plazas_totales, Plazas_disponibles) "
7             + "VALUES ((SELECT Id_Categoria_Evento FROM categoria_evento WHERE Categoria = ?), ?, ?, ?, ?, ?)";
8         PreparedStatement ps = cn.prepareStatement(SQL);
9         ps.setString(1, EV.getCategoria());
10        ps.setString(2, EV.getTitulo());
11        ps.setDate(3, EV.getFecha());
12        ps.setString(4, EV.getUbicacion());
13        ps.setInt(5, EV.getPlazas_Totales());
14        ps.setInt(6, EV.getPlazas_Disponibles());
15
16        int filasAfectadas = ps.executeUpdate();
17        System.out.println("Filas actualizadas: " + filasAfectadas);
18    }
```

Mostrar

Para el botón de Mostrar nada más creamos un array del objeto Evento que luego te comentare y lo igualamos al método ver evento de la conexión a la BD pasandole el jTable1.

```
1 private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_jButton3ActionPerformed
2
3
4     array_Eventos = objCont.Ver_Evento(jTable1);
5
6
7 }
```

```
1 public ArrayList<Evento> Ver_Evento(JTable jt) {
2
3     ArrayList<Evento> arrayList_Eventos = new ArrayList<>();
4
5     try {
6
7         DefaultTableModel vaciar_tabla = (DefaultTableModel) jt.getModel();
8         vaciar_tabla.setNumRows(0);
9         jt.setModel(vaciar_tabla);
10
11         Class.forName("com.mysql.cj.jdbc.Driver");
12         cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/bloop", "root", "");
13
14         String SQL = "SELECT "
15             + "e.Id_evento, e.Titulo, e.Fecha, e.Ubicacion, e.Plazas_totales, e.Plazas_disponibles, "
16             + "c.Categoria AS Nombre_Categoria "
17             + "FROM Evento e "
18             + "JOIN Categoria_evento c ON e.Id_Categoria_Evento = c.Id_Categoria_Evento";
19         PreparedStatement ps = cn.prepareStatement(SQL);
20         rs = ps.executeQuery();
21
22         DefaultTableModel modelo = (DefaultTableModel) jt.getModel();
23         while (rs.next()) {
24             Evento objEvento = new Evento();
25
26             objEvento.setID(rs.getInt("Id_evento"));
27             objEvento.setCategoria(rs.getString("Nombre_Categoria"));
28             objEvento.setTitulo(rs.getString("Titulo"));
29             objEvento.setFecha(rs.getDate("Fecha"));
30             objEvento.setUbicacion(rs.getString("Ubicacion"));
31             objEvento.setPlazas_Totales(rs.getInt("Plazas_totales"));
32             objEvento.setPlazas_Disponibles(rs.getInt("Plazas_disponibles"));
33
34             arrayList_Eventos.add(objEvento);
35
36             modelo.addRow(new Object[]{
37                 rs.getInt("Id_evento"),
38                 rs.getString("Nombre_Categoria"),
39                 rs.getString("Titulo"),
40                 rs.getDate("Fecha"),
41                 rs.getString("Ubicacion"),
42                 rs.getInt("Plazas_totales"),
43                 rs.getInt("Plazas_disponibles")
44             });
45         }
46     } catch (ClassNotFoundException ex) {
47         Logger.getLogger(Conect_BD.class.getName()).log(Level.SEVERE, null, ex);
48     } catch (SQLException ex) {
49         Logger.getLogger(Conect_BD.class.getName()).log(Level.SEVERE, null, ex);
50     }
51     return arrayList_Eventos;
52 }
53
54
55 }
```

En el método ver que le pedimos un JTable, hacemos la conexión y realizamos la consulta la inicializamos en la BD ejecutamos con executeQuery y el resultado se nos guarda en la variable rs que es un ResultSet. En el While con condicion rs.next (nos devuelve un true si hay otra fila, en el caso de que no haya otra fila nos devuelve false) hacemos un objeto de Evento lo rellenamos y lo introducimos en el array creados arriba que será el que devolveremos

y debajo rellenos el modelo para el JTable que previamente hemos vaciado.

Click al JTable

Al darle a una fila de las que están mostradas en el Jtable se mostrará en jText, jCombo, jSpinner de esta manera para poder editarlo y lo hacemos con el array que le pasamos al mostrar la información gracias a al `getSelectedRow`

```
1 private void jTable1MouseClicked(java.awt.event.MouseEvent evt) {GEN-FIRST:event_jTable1MouseClicked
2
3     jComboBox1.setSelectedItem(array_Eventos.get(jTable1.getSelectedRow()).getCategoria());
4     jTextField2.setText(array_Eventos.get(jTable1.getSelectedRow()).getTitulo());
5     jTextField3.setText(String.valueOf(array_Eventos.get(jTable1.getSelectedRow()).getFecha()));
6     jTextField4.setText(array_Eventos.get(jTable1.getSelectedRow()).getUbicacion());
7     jSpinner1.setValue(array_Eventos.get(jTable1.getSelectedRow()).getPlazas_Totales());
8
9 }
```

Editar

En el botón de Editar hemos creado un objeto de la clase evento donde lo rellenamos con la información que el administrador coloca al seleccionar una fila de la tabla y edita, también añadimos el id que lo sacamos del array para saber el evento al cual editamos.

Ejecutamos el método Editar con el objeto creado anteriormente.

```
1 private void jButton8ActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_jButton8ActionPerformed
2     Evento obj_eventos = new Evento();
3
4     obj_eventos.setID(array_Eventos.get(jTable1.getSelectedRow()).getID());
5     obj_eventos.setCategoria(jComboBox1.getSelectedItem().toString());
6     obj_eventos.setTitulo(jTextField2.getText());
7     obj_eventos.setFecha(Date.valueOf(jTextField3.getText()));
8     obj_eventos.setUbicacion(jTextField4.getText());
9     obj_eventos.setPlazas_Totales((Integer) jSpinner1.getValue());
10    obj_eventos.setPlazas_Disponibles(array_Eventos.get(jTable1.getSelectedRow()).getPlazas_Disponibles());
11
12
13    objCont.Editar_Evento(obj_eventos);
14 }
```

En el método pedimos un objeto de evento.

Iniciamos la conexión a la base de datos, y realizamos la consulta de Update con la misma subconsulta para conseguir el id de la categoría, además de realizar el Where para saber el evento al cual editamos.

Introducimos la información en sus respectivas posiciones dependiendo de donde este el ? antes introducimos la SQL en la base de datos y ejecutamos con `executeUpdate` (aquí yo lo meto en un int para ver las filas que están afectadas esto viene bien para ver si hay algún error y alomejor se crean 2).

```

1 public void Editar_Evento(Evento EV) {
2     try {
3         Class.forName("com.mysql.cj.jdbc.Driver");
4         cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/bloop", "root", "");
5
6         String SQL = "UPDATE evento SET "
7             + "Id_Categoria_Evento = (SELECT Id_Categoria_Evento FROM categoria_evento WHERE Categoria = ?), "
8             + "Titulo = ?, "
9             + "Fecha = ?, "
10            + "Ubicacion = ?, "
11            + "Plazas_totales = ?, "
12            + "Plazas_disponibles = ? "
13            + "WHERE Id_evento = ?";
14
15        PreparedStatement ps = cn.prepareStatement(SQL);
16
17        ps.setString(1, EV.getCategoria());
18        ps.setString(2, EV.getTitulo());
19        ps.setDate(3, EV.getFecha());
20        ps.setString(4, EV.getUbicacion());
21        ps.setInt(5, EV.getPlazas_Totales());
22        ps.setInt(6, EV.getPlazas_Disponibles());
23        ps.setInt(7, EV.getID());
24
25        int filasAfectadas = ps.executeUpdate();
26        System.out.println("Filas actualizadas: " + filasAfectadas);

```

Eliminar

En nuestro botón de eliminar funciona mediante un JOptionPane que le pedimos el id y luego una serie de casos de confirmación para que no tengamos cualquier tipo de error a la hora de eliminar un evento.

Dentro de el if que la opción es yes que confirma después de comprobar que no haya cancelación ejecutamos el método Eliminar Evento con el id que pedimos y hacemos que JButton3 que es el de mostrar se ejecute para actualizar la tabla con doClick.

```

1 private void jButton12ActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_jButton12ActionPerformed
2     String numero = JOptionPane.showInputDialog(null, "Ingresa el ID del evento a eliminar:", "Entrada de número", JOptionPane.QUESTION_MESSAGE);
3
4     if (numero != null) { // ni no cancela la acción anterior
5
6         int opcion = JOptionPane.showConfirmDialog(null, "¿Estás seguro de que quieres continuar?", "Confirmación", JOptionPane.YES_NO_OPTION);
7
8         if (opcion == JOptionPane.YES_OPTION) {
9             // Acción a realizar si dice "Si"
10            JOptionPane.showMessageDialog(null, "ID del evento a eliminar: " + numero + "\n¡Acción realizada!", "Éxito", JOptionPane.INFORMATION_MESSAGE);
11            objCont.Eliminar_Evento(Integer.parseInt(numero));
12            jButton3.doClick();
13        } else {
14            // Si dice "No"
15            JOptionPane.showMessageDialog(null, "Acción cancelada.", "Cancelado", JOptionPane.INFORMATION_MESSAGE);
16        }
17    }
18 }

```

En el método Eliminar pedimos un int que es el id que pedimos, iniciamos la conexión a la base de datos, preparamos la consulta con la condición de el id, la introducimos a la base de datos y ejecutamos con executeUpdate (aquí yo lo meto en un int para ver las filas que están afectadas esto viene bien para ver si hay algún error y alomejor se crean 2).

```

1 public void Eliminar_Evento(int id) {
2     try {
3         Class.forName("com.mysql.cj.jdbc.Driver");
4         cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/bloop", "root", "");
5
6         String SQL = "DELETE FROM evento WHERE Id_evento = " + id + ";";
7         PreparedStatement ps = cn.prepareStatement(SQL);
8
9         int filasAfectadas = ps.executeUpdate();
10        System.out.println("Filas actualizadas: " + filasAfectadas);

```

Ver Categoría



En JComboBox sale los nombres de las categorías que hay en nuestra base de datos, esto lo hacemos con una conexión a la base de datos .

Esto lo hacemos al darle al botón para abrir el panel de eventos para que se realiza de forma automática

```

1 private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_jButton1ActionPerformed
2     PanelEventos.setVisible(true);
3     objCont.Ver_Categoria(jComboBox1);
4
5 }

```

```

1 public void Ver_Categoria(JComboBox jc) {
2     try {
3         Class.forName("com.mysql.cj.jdbc.Driver");
4         cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/bloop", "root", "");
5
6         String SQL = "SELECT Categoria FROM categoria_evento";
7         PreparedStatement ps = cn.prepareStatement(SQL);
8         rs = ps.executeQuery();
9         jc.removeAllItems(); // limpia el combo antes de agregar nuevos elementos
10
11         while (rs.next()) {
12             String categoria = rs.getString("Categoria");
13             jc.addItem(categoria);
14         }
15 }

```

Para ello le pedimos es JComboBox para dentro del método rellenarlo, mediante a una consulta a categoría recibiendo el nombre.

Crear Categoría

Para el botón de crear nueva categoria le damos a él + al lado de la categoría que



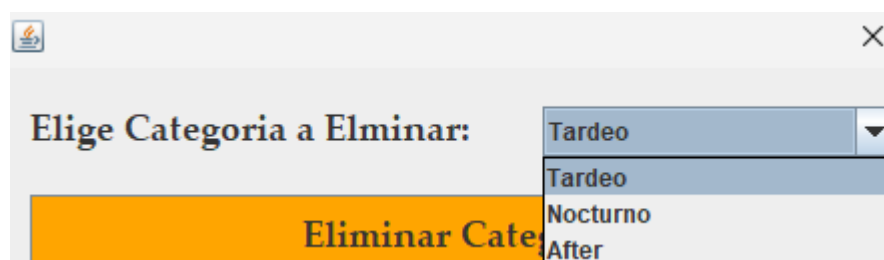
no habre un JDialog, que nos pedirá que pongamos una categoría y al darle al botón de añadir, que es un método que no pide el jText que introduce el nombre y realiza una consulta de INSERT de crear una nueva categoría.

```
1 public void Añadir_Categoria(JTextField jt) {
2     try {
3         Class.forName("com.mysql.cj.jdbc.Driver");
4         cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/bloop", "root", "");
5
6         String SQL = "INSERT INTO categoria_evento (Categoria) VALUES (?)";
7         PreparedStatement ps = cn.prepareStatement(SQL);
8         ps.setString(1, jt.getText());
9
10        ps.executeUpdate();
11    }
```

Añadiendo también el Ver categoría en el JComboBox principal para así actualizarse

```
1 private void jButton4ActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_jButton4ActionPerformed
2     objCont.Añadir_Categoria(jTextField1);
3     objCont.Ver_Categoria(jComboBox1);
4     AñadirCategoria.setVisible(false);
5 }
```

Eliminar Categoría



En el de eliminar es más de mismo al darle al - se abre el jDialog más el ver pero en este caso en el JComboBox del jDialog para así ver

los actuales, seleccionas el requerido y seleccione el botón.

En el método nos pide el String que es la selección del desplegable y realiza la consulta de DELETE mediante el nombre de la categoría, y realizamos el ver categoría del desplegable de la pantalla principal para actualizarla.

```
1 public void Eliminar_Categoria(String Categoria) {
2
3     try {
4         Class.forName("com.mysql.cj.jdbc.Driver");
5         cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/bloop", "root", "");
6
7         String SQL = "DELETE FROM categoria_evento WHERE Categoria like '" + Categoria + "'";
8         PreparedStatement ps = cn.prepareStatement(SQL);
9
10        int filasAfectadas = ps.executeUpdate();
11    }
12 }
```

Panel Reservas

En el Panel Reserva nada más nos muestra la información de todas las reservas realizadas por los usuarios.

Esto lo haces al seleccionar el Panel de Reservas se abre un JDialog y ejecutamos el código de mostrar reservas, pidiendo el jTable en el método igual que ver evento.

ID	Usuario	Evento	Tipo De Reserva	Fecha De Reserva	Nº Entradas
1	user	Tardeo de Kembo	Lol	2025-05-16	1
2	user	Tardeo de Kembo	Lol	2025-05-16	1
3	user	Tardeo de Kembo	Lol	2025-05-16	1
4	user	Tardeo de Kembo	Lol	2025-05-16	1
5	user	Tardeo de Kembo	Lol	2025-05-17	1
6	user	Tardeo de Kembo	Lol	2025-05-17	1
7	user	Tardeo de Kembo	Lol	2025-05-17	1
8	user	Tardeo de Kembo	Lol	2025-05-17	1
9	user	Tardeo de Kembo	Lol	2025-05-17	1
10	user	Tardeo de Kembo	Lol	2025-05-17	1
11	user	Tardeo de Kembo	Lol	2025-05-17	1
12	user	Tardeo de Kembo	Lol	2025-05-17	1
13	user	Tardeo de Kembo	Lol	2025-05-17	1
14	user	Tardeo de Kembo	Lol	2025-05-17	1
15	user	Tardeo de Kembo	Lol	2025-05-17	1
16	user	Tardeo de Kembo	Lol	2025-05-17	1
17	user	Tardeo de Kembo	Lol	2025-05-17	1
18	user	Tardeo de Kembo	Lol	2025-05-17	1
19	user	Tardeo de Kembo	Lol	2025-05-17	1
20	user	Tardeo de Kembo	Lol	2025-05-17	1
21	user	Tardeo de Kembo	Lol	2025-05-17	1
22	user	Tardeo de Kembo	Lol	2025-05-17	1
23	user	Tardeo de Kembo	Lol	2025-05-17	1
24	user	Tardeo de Kembo	Lol	2025-05-17	1
25	user	Tardeo de Kembo	Lol	2025-05-17	1
26	user	Tardeo de Kembo	Lol	2025-05-17	1

Panel Usuario

Este panel está destinado, para los usuarios generales, los que van a comprar entradas para los eventos disponibles. Después de iniciar sesión nos saldrá la siguiente pantalla:

Hola, usuario1

Nombre	Categoria	Fecha	Ubicacion	Plazas Disponibles
Tardeo Primavera	Tardeo	2025-06-01	Terraza Centro	78
Concierto Pop	Concierto	2025-06-10	Auditorio Norte	150
Charla Motivacional	Charla	2025-06-15	Sala Roja	60
Taller de Fotografía	Taller	2025-06-20	Aula Creativa	2

Reserva:

Tipo de Entrada:
Normal

N1 Entradas:
0

Hacer Reserva

Ver tus reservas **Volver**

```

1 public void ID(int id) {
2     this.ID = id;
3 }

```

Para poder saber que usuario el cual ha iniciado sesión lo que hacemos es un método que iniciamos en la Pantalla de Inicio Sesión, ya que al ser otro JFrame el id de Usuario solo lo tendrá el de Inicio de Sesión, de esta manera sabemos el id y podremos jugar con este en el Panel Usuario

Método de Inicio Sesión si es Usuario.

```

1 //SI ES FALSE SIGNIFICA QUE ES USUARIO Y NOS HABRE LA VENTANA DE USUARIO
2 if (objConect_BD.resultado.equals("false")) {
3     setVisible(false);
4     obj_pantauser.setVisible(true);
5     obj_pantauser.ID(objConect_BD.ID);
6 }

```

Vemos como nos pone los eventos que hay disponibles en jTable, esto lo hacemos con un método de ver reservas al abrir mediante formWindowOpened.

```
1 private void formWindowOpened(java.awt.event.WindowEvent evt) { //GEN-FIRST:event_formWindowOpened
2     objCon.Nombre_User(ID, jLabel1 );
3
4     array_Evento = objCon.Ver_Eventos_User(jTable1);
5
6     objCon.ver_entrada(jComboBox1);
7 }
```

También añadimos directamente el tipo de entrar como lo hemos hecho en la categoría en Panel de Eventos.

```
1 public void ver_entrada(JComboBox jc) {
2     try {
3         Class.forName("com.mysql.cj.jdbc.Driver");
4         cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/bloop", "root", "");
5
6         String SQL = "SELECT Tipo FROM tipo_reserva";
7         PreparedStatement ps = cn.prepareStatement(SQL);
8         rs = ps.executeQuery();
9         jc.removeAllItems(); // Opcional: limpia el combo antes de agregar nuevos elementos
10
11         while (rs.next()) {
12             String categoria = rs.getString("Tipo");
13             jc.addItem(categoria);
14         }
15     }
```

Además vemos como si seleccionamos uno podríamos realizar una reserva, con un tipo de entrada que sería Vip o normal y luego según el número de entradas, si son varias, nos saltara una ventana emergente, para poner el nombre de los acompañantes

Hacer Reserva

En el caso de la reserva hay una peculiaridad, ya que tenemos una herencia: si se hace la reserva con más de 1 entrada, se realiza la reserva grupal.

- Para ello, primero el usuario selecciona un evento y rellena los campos correspondientes: el tipo de entrada y el número de entradas. Después de esto, el usuario pulsa el botón para realizar la reserva.
- Lo primero que hacemos es crear el objeto de reserva, que rellenamos con los datos introducidos. También creamos un String que utilizaremos para guardar los nombres de los acompañantes (en caso de ser una reserva grupal). Comprobamos si el número de entradas es mayor o igual a 2. Si es así, mostramos los JPanel para pedir los nombres, los cuales guardamos en un Vector. Luego, pasamos esos nombres al String que habíamos creado.
- Después de esto, rellenamos el objeto con toda la información (incluyendo los nombres si es grupal) y se lo pasamos al método hacerReserva.
- Por último, borramos el array de eventos y lo volvemos a cargar. Esto lo hacemos para que no haya interferencias visuales al recargar los eventos, ya que al hacer una reserva, las plazas disponibles van bajando automáticamente según las entradas que compre el usuario.
- También abrimos el JDialog del hilo y lo iniciamos.

```

1 private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_jButton2ActionPerformed
2 //
3     Reserva objReserva = new Reserva();
4
5     String resultado_Nombres = "";
6
7     if ((int) jSpinner1.getValue() >= 2) {
8         String[] nombres = new String[(int) jSpinner1.getValue()];
9
10        // Pedir los textos
11        for (int i = 0; i < (int) jSpinner1.getValue(); i++) {
12            nombres[i] = JOptionPane.showInputDialog("Ingresa el nombre del " + (i + 1) + " acompañante:");
13        }
14
15        for (String texto : nombres) {
16            resultado_Nombres += texto + ", ";
17        }
18    }
19
20    objReserva.setID(ID);
21    objReserva.setNumero_Entradas((int) jSpinner1.getValue());
22    objReserva.setNombres_Acompañantes(resultado_Nombres);
23    objReserva.setEvento(array_Evento.get(jTable1.getSelectedRow()).getTitulo());
24    objReserva.setTipo_Reserva(jComboBox1.getSelectedItem().toString());
25
26
27    objCon.Hacer_Reserva(objReserva, array_Evento.get(jTable1.getSelectedRow()).getID());
28
29
30    Hilos_.setVisible(true);
31
32    array_Evento.clear();
33    array_Evento = objCon.Ver_Eventos_User(jTable1);
34    jLabel2.setText("Reserva:");
35
36    Hilos objHilos = new Hilos(jProgressBar1, jLabel6, Hilos_ );
37    objHilos.start();
38
39
40 }

```

```

1 public void Hacer_Reserva(Reserva res, int _ID) {
2
3     String SQL = "";
4     try {
5         Class.forName("com.mysql.cj.jdbc.Driver");
6         cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/bloop", "root", "");
7
8         SQL = "INSERT INTO reservas (Id_usuario, Id_evento, Id_Tipo_Reserva, Fecha_Reserva, num_entradas) "
9             + "VALUES (?, "
10            + "(SELECT Id_evento FROM evento WHERE Titulo = ? LIMIT 1), "
11            + "(SELECT Id_Tipo_reserva FROM tipo_reserva WHERE Tipo = ? LIMIT 1), CURDATE(), ?)";
12
13         PreparedStatement ps = cn.prepareStatement(SQL, Statement.RETURN_GENERATED_KEYS);
14         ps.setInt(1, res.getId());
15         ps.setString(2, res.getEvento());
16         ps.setString(3, res.getTipo_Reserva());
17
18         ps.setInt(4, res.getNumero_Entradas());
19
20         int filasAfectadas = ps.executeUpdate(); // ☒ Para INSERT
21
22         if (filasAfectadas > 0) {
23             // Obtener el ID generado de la reserva
24             ResultSet generatedKeys = ps.getGeneratedKeys();
25             if (generatedKeys.next()) {
26                 int idReserva = generatedKeys.getInt(1);
27
28                 if (res.getNumero_Entradas() >= 2) { // Si es grupal, insertar acompañantes
29                     String sqlGrupal = "INSERT INTO reserva_grupal (Id_reserva, Lista_Acompañantes) VALUES (?, ?)";
30                     PreparedStatement psGrupal = cn.prepareStatement(sqlGrupal);
31                     psGrupal.setInt(1, idReserva);
32                     psGrupal.setString(2, res.getNombres_Acompañantes()); // Ejemplo: "Lucía Pérez, Andrés Gil, Sonia Ramos"
33                     psGrupal.executeUpdate();
34                     psGrupal.close();
35                 }
36             }
37             generatedKeys.close();
38         }
39
40         PreparedStatement psUpdate = cn.prepareStatement(
41             "UPDATE Evento SET Plazas_disponibles = Plazas_disponibles - ? WHERE Id_evento = ?"
42         );
43         psUpdate.setInt(1, res.getNumero_Entradas());
44         psUpdate.setInt(2, _ID);
45         psUpdate.executeUpdate();
46

```

- En este método pedimos un objeto de reserva y un id, que es el ID del evento al cual se hace la reserva.
- Iniciamos la conexión a la base de datos, y preparamos la sentencia SQL usando Statement.RETURN_GENERATED_KEYS. Esto sirve para recuperar automáticamente los valores generados por la base de datos, como claves primarias autoincrementales (en este caso, el ID de la reserva que se acaba de crear). Metemos la información con las correspondientes ?.
- También, si el número de entradas es mayor o igual a 2, realizamos la consulta para crear la reserva grupal, utilizando el ID de la reserva que obtuvimos mediante getGeneratedKeys().
- Y por último, hacemos una consulta UPDATE que va bajando el número de plazas disponibles en el evento correspondiente.

Ver Reservas

El usuario puede ver sus reservas actuales, esto lo hacemos dándole al botón de ver reservas y nos abrirá un JDialog donde se encuentra un jTable que lo rellenamos de información.

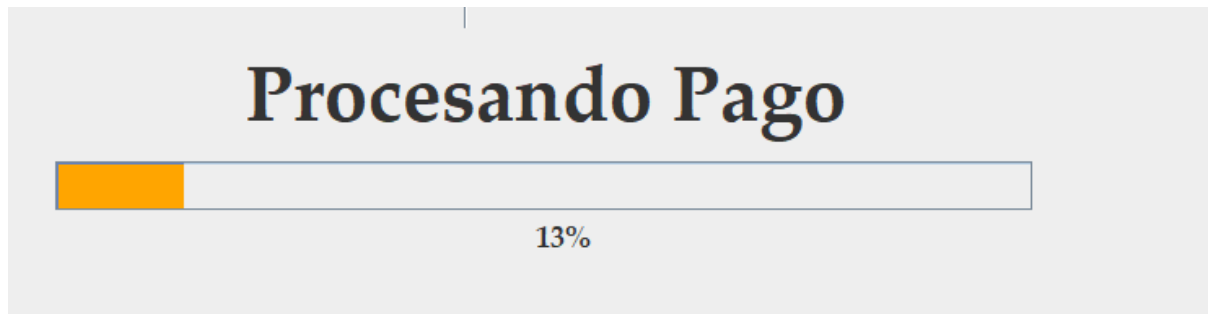
```
1 private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_jButton1ActionPerformed
2     Ver_Reservas.setVisible(true);
3     objCon.ver_Reserva(jTable2, ID);
4 }
```

En el método de ver reserva pedimos un JTable que luego rellenaremos, hacemos la conexión y realizamos la consulta la inicializamos en la BD ejecutamos con executeQuery y el resultado se nos guarda en la variable rs que es un ResultSet. En el While con condicion rs.next (nos devuelve un true si hay otra fila, en el caso de que no haya otra fila nos devuelve false) hacemos un objeto de Evento lo rellenamos y lo introducimos en el array creados arriba que será el que devolveremos y debajo rellenos el modelo para el JTable que previamente hemos vaciado.

```
1 public void Ver_Reservas(JTable jt) {
2
3     try {
4         DefaultTableModel vaciar_tabla = (DefaultTableModel) jt.getModel();
5         vaciar_tabla.setNumRows(0);
6         jt.setModel(vaciar_tabla);
7
8         Class.forName("com.mysql.cj.jdbc.Driver");
9         cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/bloop", "root", "");
10
11         String SQL = "SELECT r.Id_reserva, u.Nombre AS Nombre_Usuario, e.Titulo AS Nombre_Evento, "
12             + "t.Tipo AS Tipo_Reserva, r.Fecha_Reserva, r.num_entradas "
13             + "FROM Reservas r "
14             + "JOIN Usuario u ON r.Id_usuario = u.Id_usuario "
15             + "JOIN Evento e ON r.Id_evento = e.Id_evento "
16             + "JOIN Tipo_reserva t ON r.Id_Tipo_Reserva = t.Id_Tipo_Reserva;";
17         PreparedStatement ps = cn.prepareStatement(SQL);
18         rs = ps.executeQuery();
19
20         DefaultTableModel modelo = (DefaultTableModel) jt.getModel();
21         while (rs.next()) {
22
23             modelo.addRow(new Object[]{
24                 rs.getInt("Id_reserva"),
25                 rs.getString("Nombre_Usuario"),
26                 rs.getString("Nombre_Evento"),
27                 rs.getString("Tipo_Reserva"),
28                 rs.getDate("Fecha_Reserva"),
29                 rs.getInt("num_entradas")
30             });
31         }
32     }
33 }
```

Hilos

El hilo que hemos introducido en nuestro programa es una pantalla que sale al realizar la compra de entradas, mediante un proceso de pago.



```
1 public Hilos(JProgressBar L, JLabel label, JDialog dialogo) {
2     this.L = L;
3     this.label = label;
4     this.dialogo = dialogo;
5 }
6
7 @Override
8 public void run() {
9     int valor = L.getValue();
10
11     while (valor < 100) {
12         try {
13             int tiempoEspera = 100 + aleatorio.nextInt(900);
14             Thread.sleep(tiempoEspera);
15
16             int incremento = 1 + aleatorio.nextInt(5);
17             valor = Math.min(valor + incremento, 100);
18
19             L.setValue(valor);
20             label.setText(valor + "%");
21
22         } catch (InterruptedException ex) {
23             ex.printStackTrace();
24         }
25     }
26
27     dialogo.dispose();
28 }
29 }
```

Esto funciona mediante un hilo el cual le rellenamos con un JProgress, JLabel para la información de la pantalla y un JDialog que nos servirá para cerrar el proceso de pago al acabar el hilo. La función de esto es recoger el % del Progress que es 0 y realizar un bucle hasta que el valor sea mayor a 100 hacemos un numero aleatorio para el tiempo de espera entre cada paso del bucle para que no sea todo seguido

hacemos el Sleep de el tiempo de espera, y ahora hacemos otro número aleatorio del 1 a 6 para el % que se va a aumentar, hacemos un Math.min que significa que nos subirá el porcentaje mientras sea menor a 100, lo introducimos al Progress y Label y al terminar cerramos el Dialog.

```
1 private void WindowClosed(java.awt.event.WindowEvent evt) {GEN-FIRST:event_WindowClosed
2     progressBar1.setValue(0);
3     jLabel16.setText("0%");
4 }
```

También añadir que al cerrar el Jdialog se colocara el Valor del Progres en 0 para que al volver ejecutar se inicie otra vez.